

NSAPDEV Server Application Project

Project Specification: "Mini-Splunk" Syslog Analytics Server

1 Project Overview

This course prepares students to build robust and scalable back-end services for a wide variety of server-based applications. This project involves building a lightweight, centralized log management and analytics server designed to ingest, parse, and analyze standard **syslog** files. Paired with a Command Line Interface (CLI) client, the system will handle bulk log file uploads from multiple concurrent client instances. The server will parse the uploaded syslog files into standard components (Timestamp, Hostname, Process, Severity, Message), index the data safely in memory or disk, and allow clients to execute search, aggregation, and administrative commands.

2 Learning Outcomes Addressed

This architecture serves as a practical application of the course's Core Learning Outcomes:

- **CLO1:** To be able to determine server application requirements and model its architecture according to required use and application domain.
- **CLO2:** To understand how operating systems create and execute processes or threads to manage isolated client connections.
- **CLO3 & CLO4:** Use concurrent or parallel techniques to improve the performance of the application and handle concurrent requests for services (e.g., locks, mutexes to safely write to or clear shared data structures).

3 Core Components & Functional Requirements

3.1 The "Indexer" (Server Application)

- **Concurrent File Ingestion:** The server must use multithreading, multiprocessing, or asynchronous I/O to accept simultaneous file transfers from multiple clients without blocking other operations.
- **Syslog Parsing & Shared State:** Upon receiving a syslog file, the server must parse the lines into their constituent parts using regular expressions. The parsed data must be stored in a centralized, thread-safe structure. The server must implement appropriate isolation and locking to prevent race conditions during concurrent parsing, indexing, and purging.

3.2 The "Forwarder & Search Head" (CLI Client Application)

The CLI must allow the user to specify local syslog files to upload, type analytical commands to interrogate the remote server, and manage the server's data state. Below are the required commands and their expected behaviors:

A. File Uploading (INGEST)

- **Format:** `INGEST <file_path> <IP_or_DNS>:<Port>`
- **Behavior:** Reads a local file and streams it to the server for parsing.
- **Example Usage:**

```
client> INGEST /var/log/syslog 192.168.1.100:65432
[System Message] Connecting to 192.168.1.100:65432...
[System Message] Uploading syslog (2.1 MB)...
```

```
[Server Response] SUCCESS: File received and 18,432 syslog entries parsed and indexed.
```

B. Searching by Date (SEARCH_DATE)

- **Format:** QUERY <IP_or_DNS>:<Port> SEARCH_DATE "<date_string>"
- **Behavior:** Asks the server to return all parsed log lines that match a specific date or timestamp prefix.
- **Example Usage:**

```
client> QUERY 192.168.1.100:65432 SEARCH_DATE "Feb 22"
[System Message] Sending query...
[Server Response] Found 3 matching entries for date 'Feb 22':
1. Feb 22 00:05:38 SYSSVR1 systemd[1]: Started OpenBSD Secure Shell server...
2. Feb 22 00:05:54 SYSSVR1 systemd[1]: Started OpenBSD Secure Shell server...
3. Feb 22 00:05:57 SYSSVR1 systemd[1]: ssh@125339... Deactivated successfully.
```

C. Searching by Hostname (SEARCH_HOST)

- **Format:** QUERY <IP_or_DNS>:<Port> SEARCH_HOST <hostname>
- **Behavior:** Asks the server to return all parsed log lines originating from a specific machine.
- **Example Usage:**

```
client> QUERY 192.168.1.100:65432 SEARCH_HOST SYSSVR1
[System Message] Sending query...
[Server Response] Found 2 matching entries for host 'SYSSVR1':
1. Feb 22 00:05:38 SYSSVR1 systemd[1]: Started OpenBSD Secure Shell server...
2. Feb 22 00:05:57 SYSSVR1 systemd[1]: ssh@125339... Deactivated successfully.
```

D. Searching by Process/Daemon (SEARCH_DAEMON)

- **Format:** QUERY <IP_or_DNS>:<Port> SEARCH_DAEMON <daemon_name>
- **Behavior:** Filters the server's indexed logs to show only those generated by a specific service.
- **Example Usage:**

```
client> QUERY logserver.local:8080 SEARCH_DAEMON sshd
[System Message] Sending query...
[Server Response] Found 1 matching entry for daemon 'sshd':
1. Feb 22 01:14:22 SYSSVR1 sshd[4421]: Failed password for invalid user admin...
```

E. Searching by Severity (SEARCH_SEVERITY)

- **Format:** QUERY <IP_or_DNS>:<Port> SEARCH_SEVERITY <severity_level>
- **Behavior:** Filters the server's indexed logs to show only those matching a specific severity level (e.g., ERROR, WARN, INFO).
- **Example Usage:**

```
client> QUERY 192.168.1.100:65432 SEARCH_SEVERITY ERROR
[System Message] Sending query...
[Server Response] Found 1 matching entry for severity 'ERROR':
1. Feb 22 02:10:05 SYSSVR1 kernel: [ 1234.567890] ERROR: Disk quota exceeded
```

F. Searching by Keyword (SEARCH_KEYWORD)

- **Format:** QUERY <IP_or_DNS>:<Port> SEARCH_KEYWORD <keyword>
- **Behavior:** Scans the logs (specifically the "Message" portion) and returns the full log lines containing the exact keyword or phrase.

- **Example Usage:**

```
client> QUERY 192.168.1.100:65432 SEARCH_KEYWORD "Failed password"
[System Message] Sending query...
[Server Response] Found 1 matching entry for keyword 'Failed password':
1. Feb 22 01:14:22 SYSSVR1 sshd[4421]: Failed password for invalid user admin
from 10.0.0.9...
```

G. Counting by Keyword (COUNT_KEYWORD)

- **Format:** QUERY <IP_or_DNS>:<Port> COUNT_KEYWORD <keyword>
- **Behavior:** Searches the logs and returns an aggregate count of how many times the keyword appears, rather than the log lines themselves.
- **Example Usage:**

```
client> QUERY 192.168.1.100:65432 COUNT_KEYWORD Deactivated
[System Message] Sending query...
[Server Response] The keyword 'Deactivated' appears in 1 indexed log entry.
```

H. Erasing All Logs (PURGE)

- **Format:** PURGE <IP_or_DNS>:<Port>
 - **Behavior:** Instructs the server to safely delete all indexed log entries from its memory or disk. The server must acquire an exclusive write lock to ensure no other threads are reading or writing data while the clear operation is in progress.
 - **Example Usage:**
- ```
client> PURGE 192.168.1.100:65432
[System Message] Connecting to 192.168.1.100:65432 to purge records...
[Server Response] SUCCESS: 18,432 indexed log entries have been erased.
```

### 4 Technical Specifications

- **Implementation Language:** Python Interpreter, GNU C/C++ compiler using standard socket libraries.
- **Concurrency Model:** Must rely on native OS support for parallelism (e.g., threading, multiprocessing, or asyncio) rather than high-level web frameworks.

### 5 Deliverables & Grading

This project fulfills the Major Course Output (MCO) requirements:

- **Project Architecture (Due Week 8):** Software architecture design of the application
- **Server Application Project (Due Week 13):** working source code, server application, client application and documentation.

### 6 Course Policies

- **Intellectual Honesty:** Any requirements submitted for grading should reflect individual work effort and intellectual honesty. Group projects submitted for grading should reflect group work effort and intellectual honesty. Anybody caught cheating or violating intellectual honesty will be subject to appropriate disciplinary action.

- **Late Penalties:** All deliverables should be submitted on or before the deadline. Five points will be deducted for every day late.